

BÀI THỰC HÀNH FACADE PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Frontend: React (gọi API đơn giản)
 - ✓ Áp dụng **Facade Pattern** chuẩn GoF
 - ✓ Ẩn logic phức tạp của nhiều subsystem phía sau một interface đơn giản
-

BÀI TOÁN

Hệ thống **E-commerce** khi đặt hàng cần thực hiện nhiều bước:

- Kiểm tra tồn kho
- Xử lý thanh toán
- Gửi email xác nhận

👉 Vấn đề:

Controller hoặc Service phải gọi nhiều subsystem → **code phức tạp, khó bảo trì**

👉 Giải pháp:

Sử dụng **Facade Pattern** để gom tất cả vào **một lớp duy nhất**

📦 KIẾN TRÚC HỆ THỐNG

Frontend (React)



POST /api/orders/place



OrderFacade (Facade)



```
-----  
| InventoryService      |  
| PaymentService        |  
| NotificationService    |  
-----
```

💻 BACKEND – ASP.NET Core 8 Web API

1 Tạo project

```
dotnet new webapi -n FacadeDemo  
cd FacadeDemo
```

📁 Cấu trúc thư mục

```
FacadeDemo  
├── Controllers  
│   └── OrdersController.cs  
├── Facades  
│   └── OrderFacade.cs  
├── Services  
│   ├── InventoryService.cs  
│   ├── PaymentService.cs  
│   └── NotificationService.cs  
├── Models  
│   ├── OrderRequest.cs  
│   └── OrderResult.cs  
└── Program.cs
```

🔗 1. MODEL

Models/OrderRequest.cs

```
namespace FacadeDemo.Models;  
  
public class OrderRequest  
{  
    public string ProductName { get; set; }  
    public int Quantity { get; set; }  
    public decimal Amount { get; set; }  
    public string Email { get; set; }  
}
```

Models/OrderResult.cs

```
namespace FacadeDemo.Models;  
  
public class OrderResult
```

```
{  
    public bool Success { get; set; }  
    public string Message { get; set; }  
}
```

❧ 2. SUBSYSTEMS (HỆ THỐNG CON)

InventoryService

```
namespace FacadeDemo.Services;  
  
public class InventoryService  
{  
    public bool CheckStock(string product, int quantity)  
    {  
        Console.WriteLine("Checking inventory...");  
        return true; // giả lập luôn còn hàng  
    }  
}
```

PaymentService

```
namespace FacadeDemo.Services;  
  
public class PaymentService  
{  
    public bool ProcessPayment(decimal amount)  
    {  
        Console.WriteLine("Processing payment...");  
        return true;  
    }  
}
```

NotificationService

```
namespace FacadeDemo.Services;  
  
public class NotificationService  
{  
    public void SendEmail(string email)  
    {
```

```
        Console.WriteLine($"Sending email to {email}");
    }
}
```

❁ 3. FACADE (TRỌNG TÂM)

Facades/OrderFacade.cs

```
using FacadeDemo.Models;
using FacadeDemo.Services;

namespace FacadeDemo.Facades;

public class OrderFacade
{
    private readonly InventoryService _inventory;
    private readonly PaymentService _payment;
    private readonly NotificationService _notification;

    public OrderFacade(
        InventoryService inventory,
        PaymentService payment,
        NotificationService notification)
    {
        _inventory = inventory;
        _payment = payment;
        _notification = notification;
    }

    public OrderResult PlaceOrder(OrderRequest request)
    {
        // 1. Check tồn kho
        if (!_inventory.CheckStock(request.ProductName,
request.Quantity))
        {
            return new OrderResult
            {
                Success = false,
                Message = "Out of stock"
            };
        }
    }
}
```

```

        // 2. Thanh toán
        if (!_payment.ProcessPayment(request.Amount))
        {
            return new OrderResult
            {
                Success = false,
                Message = "Payment failed"
            };
        }

        // 3. Gửi email
        _notification.SendEmail(request.Email);

        return new OrderResult
        {
            Success = true,
            Message = "Order placed successfully"
        };
    }
}

```

✿ 4. CONTROLLER

Controllers/OrdersController.cs

```

using Microsoft.AspNetCore.Mvc;
using FacadeDemo.Facades;
using FacadeDemo.Models;

namespace FacadeDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderFacade _orderFacade;

    public OrdersController(OrderFacade orderFacade)
    {
        _orderFacade = orderFacade;
    }
}

```

```
[HttpPost("place")]
public IActionResult PlaceOrder(OrderRequest request)
{
    var result = _orderFacade.PlaceOrder(request);
    return Ok(result);
}
}
```

🔗 5. ĐĂNG KÝ DI

Program.cs

```
using FacadeDemo.Facades;
using FacadeDemo.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Subsystems
builder.Services.AddTransient<InventoryService>();
builder.Services.AddTransient<PaymentService>();
builder.Services.AddTransient<NotificationService>();

// Facade
builder.Services.AddTransient<OrderFacade>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});

var app = builder.Build();
```

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.UseCors("AllowReact");

app.MapControllers();
app.Run();
```

CHẠY BACKEND

dotnet run

Swagger:

<https://localhost:xxxx/swagger>

Test API:

POST /api/orders/place

Body:

```
{
  "productName": "Laptop",
  "quantity": 1,
  "amount": 1000,
  "email": "user@gmail.com"
}
```

FRONTEND – REACT

Tạo project

```
npx create-react-app facade-ui
cd facade-ui
npm install axios
```

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [result, setResult] = useState("");

  const placeOrder = async () => {
    const response = await axios.post(
      "https://localhost:xxxx/api/orders/place",
      {
        productName: "Laptop",
        quantity: 1,
        amount: 1000,
        email: "user@gmail.com",
      }
    );

    setResult(response.data.message);
  };

  return (
    <div style={{ padding: 40 }}>
      <h2>Place Order</h2>

      <button onClick={placeOrder}>Order Now</button>

      <h3>{result}</h3>
    </div>
  );
}

export default App;
```

▶ CHẠY FRONTEND

npm start

LUỒNG XỬ LÝ

Khi user đặt hàng:

React gửi request:

```
{
  "productName": "Laptop",
  "quantity": 1,
  "amount": 1000,
  "email": "user@gmail.com"
}
```

Backend xử lý:

```
OrdersController
    ↓
OrderFacade
    ↓
1. InventoryService.CheckStock()
2. PaymentService.ProcessPayment()
3. NotificationService.SendEmail()
```

SO SÁNH TRƯỚC & SAU FACADE

✗ Không dùng Facade

Controller phải gọi:

```
_inventory.CheckStock();
_payment.ProcessPayment();
_notification.SendEmail();
```

👉 Code rối, phụ thuộc nhiều

✓ Dùng Facade

```
_orderFacade.PlaceOrder(request);
```

👉 Gọn – sạch – dễ mở rộng

🎓 MỤC TIÊU SINH VIÊN ĐẠT ĐƯỢC

- Hiểu rõ **Facade Pattern** dùng để làm gì
 - Phân biệt:
 - Facade vs Service thường
 - Biết cách:
 - Gom nhiều subsystem thành 1 API đơn giản
 - Áp dụng trong:
 - E-commerce
 - Microservices orchestration
 - Clean Architecture
-

🚀 BÀI TẬP MỞ RỘNG

1. Thêm:
 - ShippingService (giao hàng)
2. Thêm:
 - Logging
3. Tách:
 - Payment thành nhiều loại (kết hợp Factory Pattern)
4. Viết:
 - Unit Test cho Facade